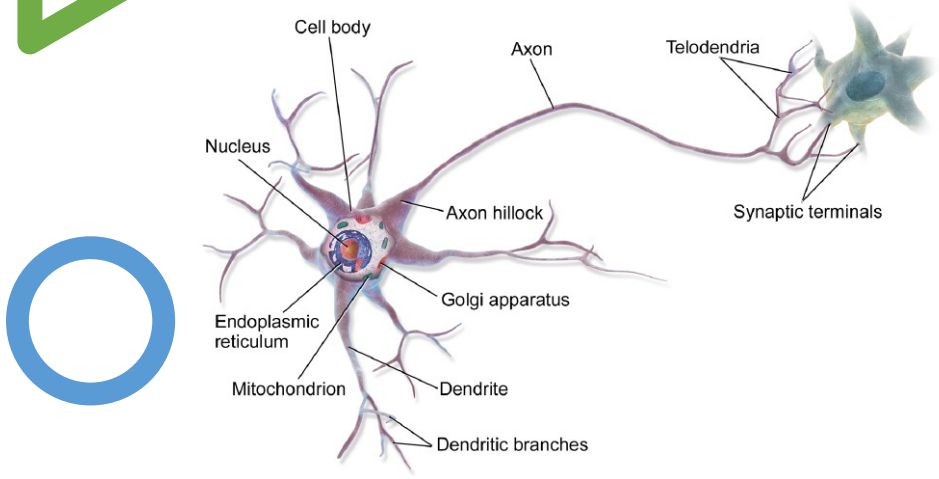
The Perceptron

From Chapter 10 of the book “Hands-on Machine Learning”, A. Geron.



ANN

- Nature has inspired countless inventions:
  - Birds inspired us to fly
  - Burdock plants inspired Velcro

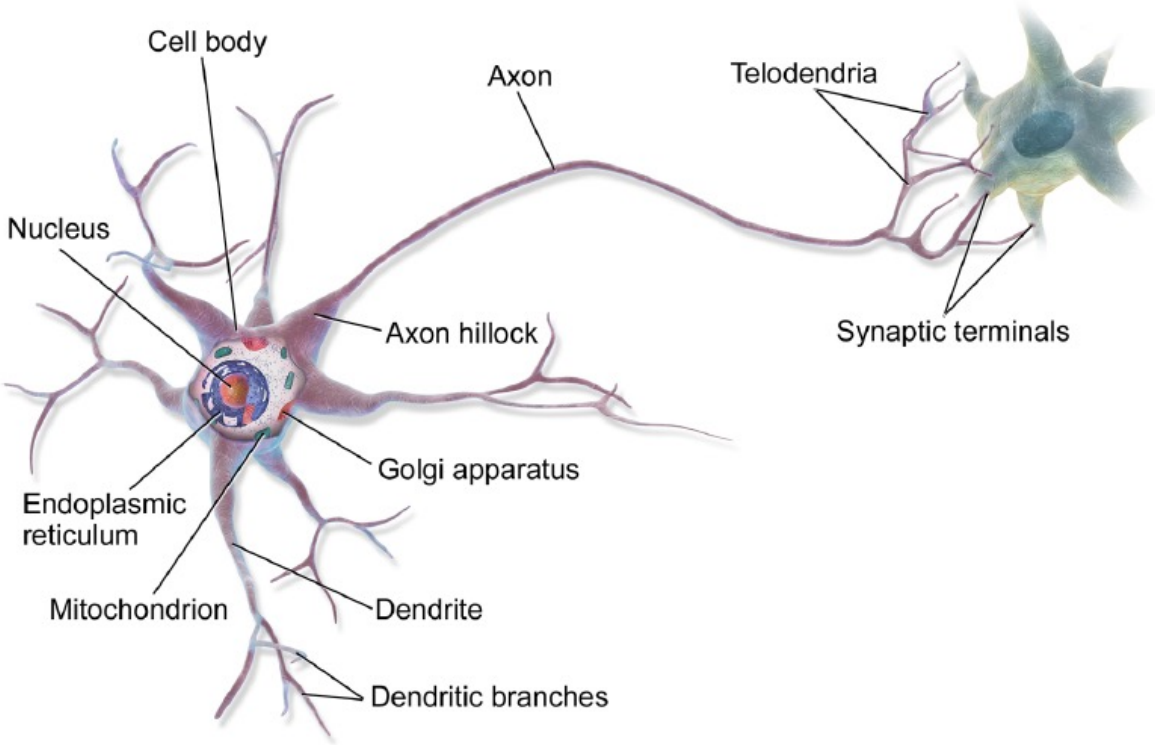
- ***Artificial Neural Networks (ANNs)*** are machine learning models inspired by the networks of biological neurons in our brains.
- Although planes were inspired by birds, they don't have to flap their wings.
- Similarly, ANNs have gradually become quite different from their biological cousins.



Burdock Plant

Biological Neurons

- **Cell body:** containing the nucleus and most of the cell's complex components,
- **Dendrites**: many branching extensions
- **Axon**: one very long extension, The axon's length may be just a few times longer than the cell body, or up to tens of thousands of times longer.
- **Telodendria:** Near its extremity the axon splits off into many branches called, and at the tip of these branches are minuscule structures called **synaptic terminals** (or simply **synapses**), which are connected to the dendrites or cell bodies of other neurons.
- Biological neurons produce short electrical impulses called **action potentials** (APs, or just signals) which travel along the axons and make the synapses release chemical signals called **neurotransmitters**.
- When a neuron receives a sufficient amount of these neurotransmitters within a few milliseconds, it fires its own electrical impulses (actually, it depends on the neurotransmitters, as some of them inhibit the neuron from firing).



Biological Neurons

- Biological neurons seem to behave in a rather simple way, but they are organized in a vast network of billions, with each neuron typically connected to thousands of other neurons.

- Highly complex computations can be performed by a network of fairly simple neurons,
  - Much like a complex anthill can emerge from the combined efforts of simple ants.



BNNs

- The architecture of **biological neural networks (BNNs)** is still the subject of active research
- Some parts of the brain have been mapped
- Neurons are often organized in consecutive layers, especially in the cerebral cortex (i.e., the outer layer of your brain), as shown in the Figure.

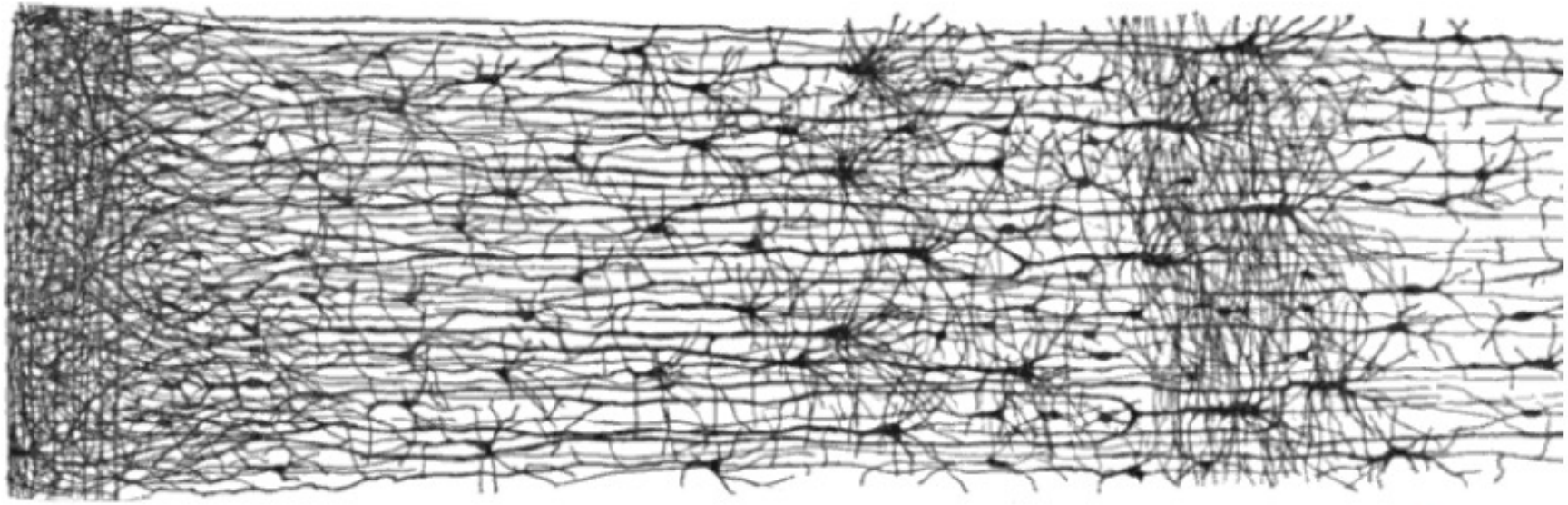


Figure 10-2. Multiple layers in a biological neural network (human cortex)[6]

Logical Computations with Neurons

- McCulloch and Pitts proposed a very simple model of the biological neuron, which later became known as an ***artificial neuron***:
  - It has one or more binary (on/off) inputs and one binary output.
  - The artificial neuron activates its output when more than a certain number of its inputs are active.
- In their paper, they showed that even with such a simplified model it is possible **to build a network of artificial neurons that computes any logical proposition you want**.
- To see how such a network works, let's build a few ANNs that perform various logical computations (see Figure 10-3), *assuming* that:
  - A neuron is activated when at least two of its inputs are active.

Logical Computations with Neurons

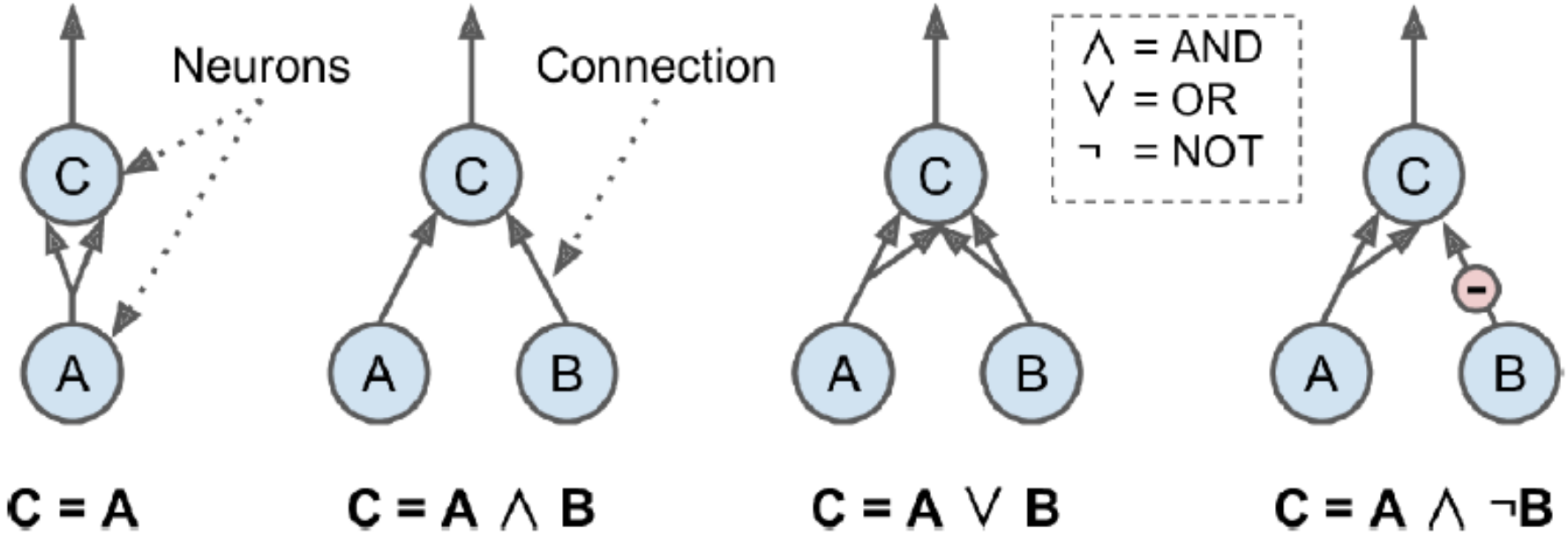


Figure 10-3. ANNs performing simple logical computations

Assumption: A neuron is activated when at least two of its inputs are active.

Logical Computations with Neurons

- The first network on the left is the **identity function**: if neuron A is activated, then neuron C gets activated as well (since it receives two input signals from neuron A); but if neuron A is off, then neuron C is off as well.
- The second network performs a **logical AND**: neuron C is activated only when both neurons A and B are activated (a single input signal is not enough to activate neuron C).
- The third network performs a **logical OR**: neuron C gets activated if either neuron A or neuron B is activated (or both).

Logical Computations with Neurons

- Finally, if we suppose that an input connection can inhibit the neuron's activity (which is the case with biological neurons), then the fourth network computes a slightly more complex logical proposition: neuron C is activated only if neuron A is active and neuron B is off.
  - If neuron A is active all the time, then you get a **logical NOT**: neuron C is active when neuron B is off, and vice versa.
- You can imagine how these networks can be combined to compute complex logical expressions.

The Perceptron

- The Perceptron is one of the simplest ANN architectures, invented in 1957 by Frank Rosenblatt.
- It is based on a slightly different artificial neuron called a **threshold logic unit (TLU)**, or sometimes a **linear threshold unit (LTU)**.
- The inputs and output are **numbers** (instead of binary on/off values), and each input connection is associated with a *weight*.
- The TLU computes a weighted sum of its inputs

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w}^\intercal \mathbf{x} + b$$

- then applies a step function to that sum and outputs the result:

$h_w(\mathbf{x}) = \text{step}(z)$,

The TLU

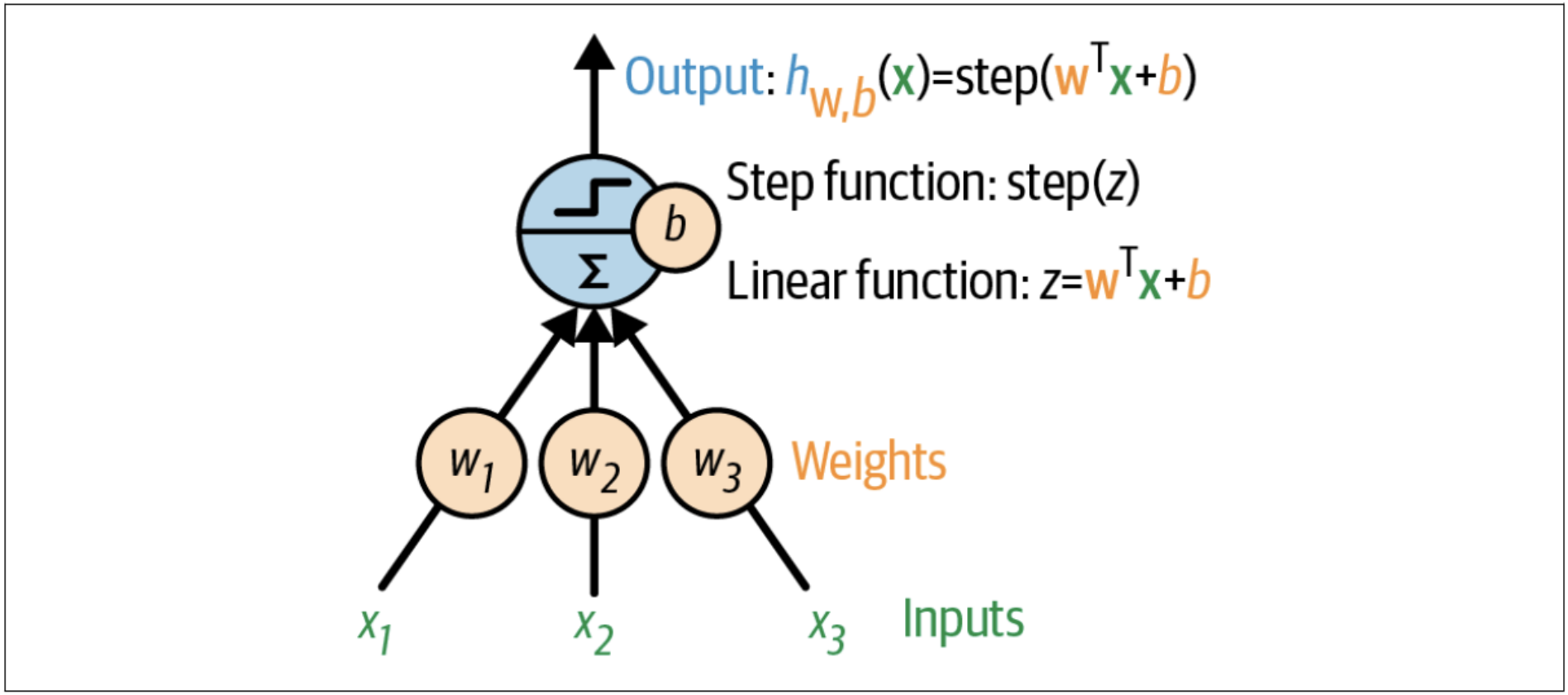


Figure 10-4. TLU: an artificial neuron that computes a weighted sum of its inputs $\mathbf{w}^T \mathbf{x}$*, plus a bias term b, then applies a step function*

The Perceptron

- The most common step function used in Perceptrons is the *Heaviside step function*. Sometimes the sign function is used instead.

Equation 10-1. Common step functions used in Perceptrons (assuming threshold = 0)

$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases} \qquad \text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

The TLU

- A **single** TLU can be used for simple **linear binary classification:**
  - It computes a linear combination of the inputs, and if the result exceeds a threshold, it outputs the positive class.
  - Otherwise, it outputs the negative class (just like a Logistic Regression or linear SVM classifier).
- You could, use a single TLU to classify iris flowers based on petal length and width (also adding an extra bias feature $x_0=1$).
- Training a TLU in this case means finding the right values for w_0, w_1, and w_2 (the training algorithm is discussed shortly).

The Perceptron

- A Perceptron is composed of one or more TLUs organized in a **single layer of TLUs**, with each TLU connected to all the inputs.
- When all the neurons in a layer are connected to every neuron in the previous layer (i.e., its input neurons), the layer is called a **fully connected layer**, or a **dense layer**.
- The inputs of the Perceptron are fed to special passthrough neurons called input neurons: they output whatever input they are fed.
- All the input neurons form the **input layer**. Moreover, an extra bias feature is generally added ($x_0 = 1$): it is typically represented using a special type of neuron called a **bias neuron**, which outputs 1 all the time.
- A Perceptron with two inputs and three outputs is represented in Figure 10-5.
- This Perceptron can classify instances simultaneously into three different binary classes, which makes it a **multioutput classifier**.

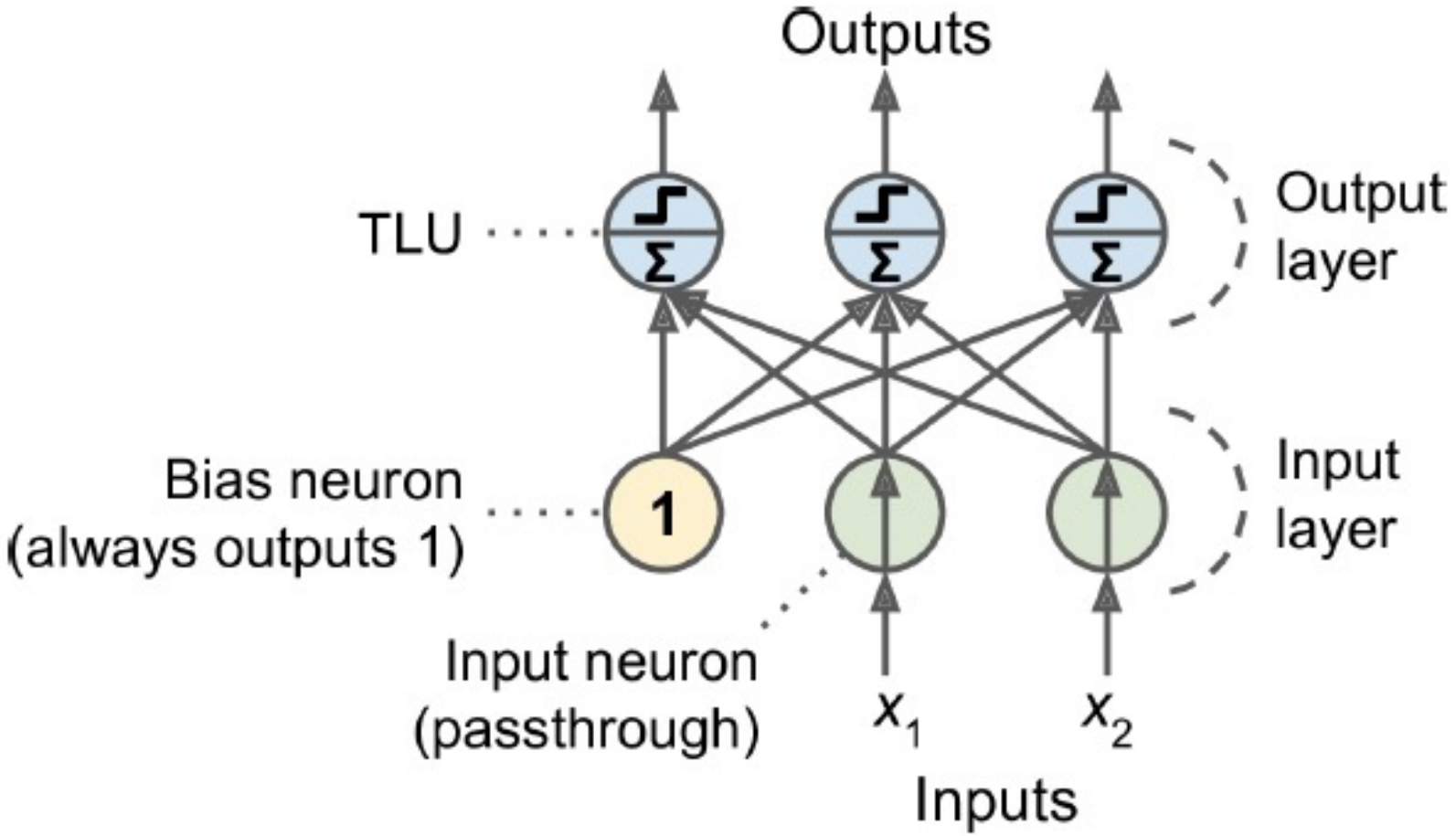


Figure 10-5. Architecture of a Perceptron with two input neurons, one bias neuron, and three output neurons

The Perceptron

- It possible to efficiently compute the outputs of a layer of artificial neurons for several instances at once.
- As always, **X** represents the matrix of input features. It has one row per instance and one column per feature.
- The weight matrix **W** contains all the connection weights except for the ones from the bias neuron. It has one row per input neuron and one column per artificial neuron in the layer.
- The bias vector **b** contains all the connection weights between the bias neuron and the artificial neurons. It has one bias term per artificial neuron.
- The function ϕ is called the *activation function*: when the artificial neurons are TLUs, it is a step function (but we will discuss other activation functions shortly).

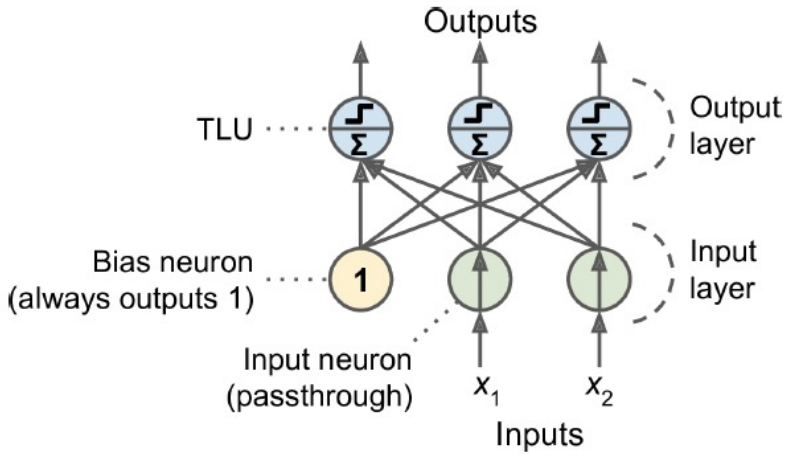


Figure 10-5. Architecture of a Perceptron with two input neurons, one bias neuron, and three output neurons

$$h_{\mathbf{W},\mathbf{b}}(\mathbf{X}) = \phi(\mathbf{X}\mathbf{W} + \mathbf{b})$$

How is a Perceptron trained?

- The Perceptron training algorithm proposed by Rosenblatt was largely inspired by Hebb's rule. In his 1949 book The Organization of Behavior (Wiley), Donald Hebb suggested that when a biological neuron triggers another neuron often, the connection between these two neurons grows stronger.
- Siegrid Löwel later summarized Hebb's idea in the catchy phrase, "**Cells that fire together, wire together**"; that is, the connection weight between two neurons tends to increase when they fire simultaneously.
- This rule later became known as Hebb's rule (or **Hebbian learning**).
- Perceptrons are trained using a variant of this rule that takes into account the error made by the network when it makes a prediction; the **Perceptron learning rule** reinforces connections that help reduce the error.
- More specifically, the Perceptron is fed one training instance at a time, and for each instance it makes its predictions.
- For every output neuron that produced a wrong prediction, it reinforces the connection weights from the inputs that would have contributed to the correct prediction.

Perceptron Training

Equation 10-3. Perceptron learning rule (weight update)

$$w_{i,j}^{(\text{next step})} = w_{i,j} + \eta \left(y_j - \hat{y}_j \right) x_i$$

In this equation:

- $w_{i,j}$ is the connection weight between the ith input neuron and the jth output neuron.
- x_i is the ith input value of the current training instance.
- $\hat{y}_j$ is the output of the jth output neuron for the current training instance.
- y_j is the target output of the jth output neuron for the current training instance.
- η is the learning rate.

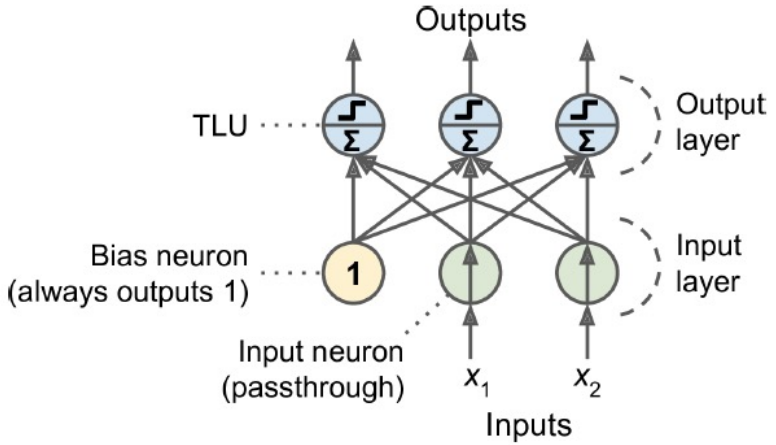


Figure 10-5. Architecture of a Perceptron with two input neurons, one bias neuron, and three output neurons

Perceptron Training

- The decision boundary of each output neuron is linear, so Perceptrons are incapable of learning complex patterns (just like Logistic Regression classifiers).
- However, if the training instances are linearly separable, Rosenblatt demonstrated that this algorithm would converge to a solution.
- This is called the Perceptron convergence theorem.

Perceptron: Scikit-Learn

- Scikit-Learn provides a Perceptron class that implements a single-TLU network.
- It can be used pretty much as you would expect—for example, on the iris dataset:
- **Iris dataset**: sepal and petal length and width of 150 iris flowers of three species: *Iris setosa*, *Iris versicolor* and *Iris virginica*.

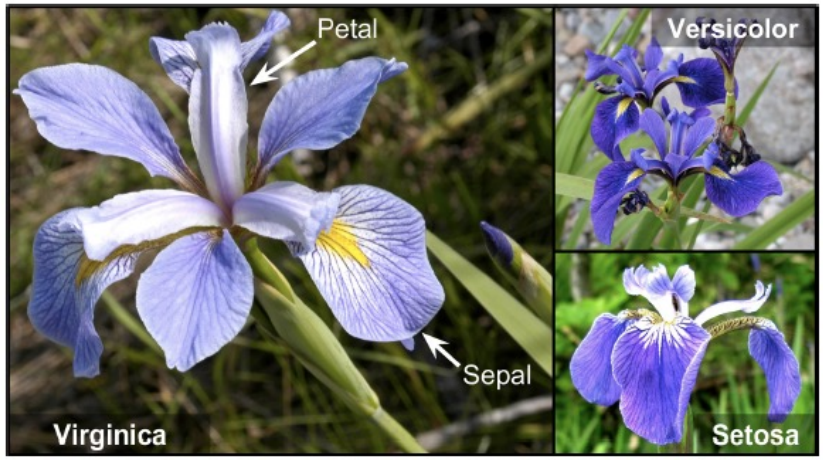


Figure 4-22. Flowers of three iris plant species[14]

Perceptron: Scikit-Learn

- You may have noticed that the Perceptron learning algorithm strongly resembles Stochastic Gradient Descent.
- In fact, Scikit-Learn's **`Perceptron`** class is equivalent to using an **`SGDClassifier`** with the following hyperparameters: **`loss="perceptron" learning_rate="constant", eta0=1`** (the learning rate), and **`penalty=None`** (no regularization).

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris(as_frame=True)
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 0)  # Iris setosa

per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)

X_new = [[2, 0.5], [3, 1]]
y_pred = per_clf.predict(X_new)  # predicts True and False for these 2 flowers
```

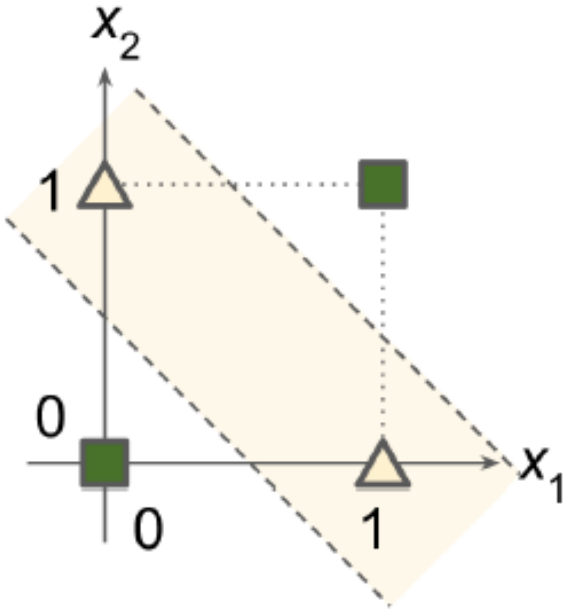
Perceptron

- Note that contrary to Logistic Regression classifiers, Perceptrons do not output a class probability; rather, they make predictions based on a hard threshold.

- This is one reason to prefer Logistic Regression over Perceptrons.

Perceptrons: Limitations

- Perceptrons are incapable of solving some trivial problems (e.g., the Exclusive OR (XOR) classification problem.
- This is true of any other linear classification model (such as Logistic Regression classifiers).



MLP: Multilayer Perceptron

- It turns out that some of the limitations of Perceptrons can be eliminated by stacking multiple Perceptrons.
- The resulting ANN is called a Multilayer Perceptron (MLP).
- An MLP can solve the XOR problem, as you can verify by computing the output of the MLP: with inputs (0, 0) or (1, 1), the network outputs 0, and with inputs (0, 1) or (1, 0) it outputs 1.
- All connections have a weight equal to 1, except the four connections where the weight is shown.
- Try verifying that this network indeed solves the XOR problem!

